
Software Development Lifecycle: Process Models

Software Process Models

- **Process model or SE paradigm**

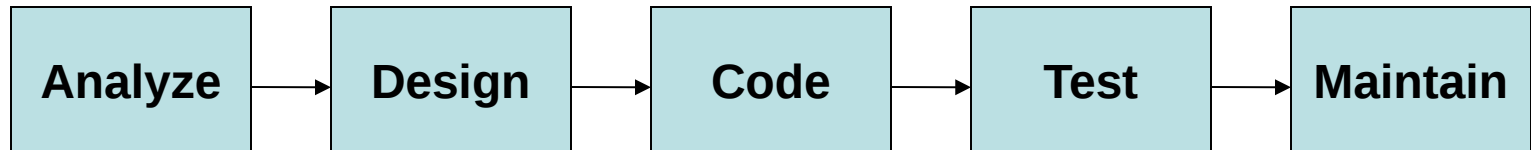
A strategy to be incorporated by an engineer or a team of engineers that encompasses the process, methods and tools involved in development of software.

- Chosen based on the nature of application involved, available methods, tools and engineers and the deliverables that are required.

Software Process Models

- Linear sequential model
- Prototyping model
- Rapid application development model
- Evolutionary software process models:
 - Incremental model
 - Spiral model
 - Concurrent development model
- Component based development
- Model based development

Linear Sequential Model



- One of the oldest and most widely used paradigms.
- Typically meant for software that is developed and delivered, in one go.
- Flow from one stage to another is usually not smooth.

Linear Sequential Model: Activities

- **System engineering:** Typically does not fall under SE lifecycle.
- **Requirements capture and analysis:** Requirements are captured, analyzed, documented and reviewed with customers.
- **Design:** Focuses on four aspects:
 - Software architecture
 - Algorithms
 - Data structures
 - Interfaces.Design, again, is documented, but may not always be reviewed with customers.

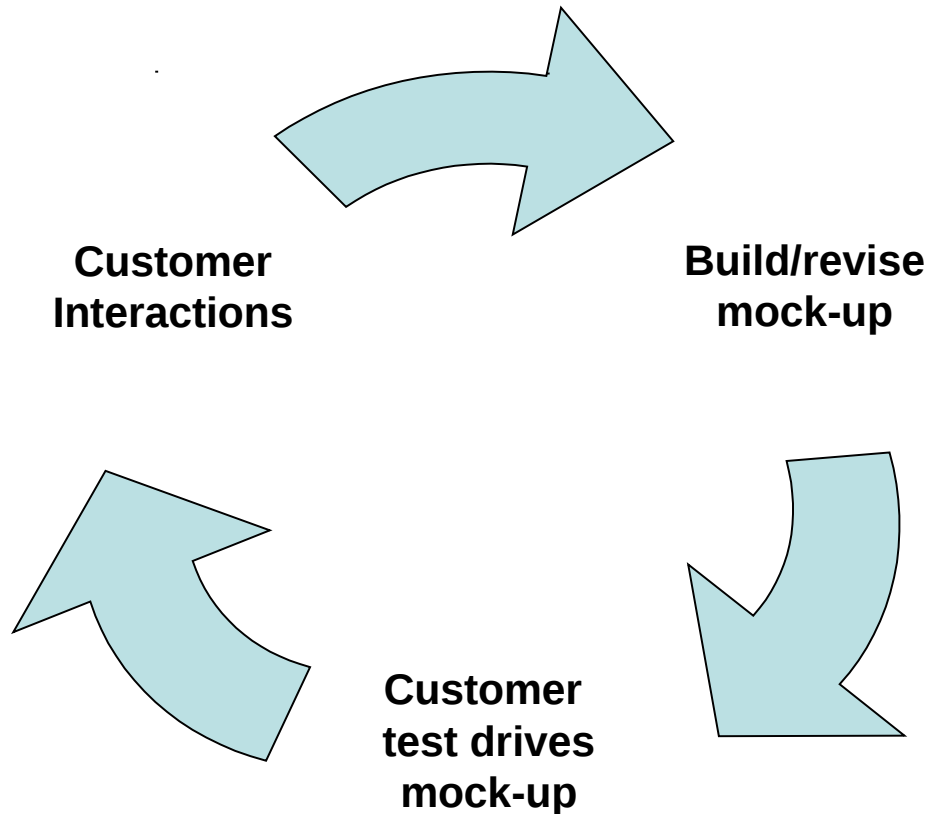
Linear Sequential Model: Activities

- **Coding:** Usually not very time consuming, if design is well done. Also involves unit testing.
- **Testing:** Involves both black and white box testing. May involve integration and system testing too.
- **Support:** Needed due to change in requirements. Any of the earlier phases can be applied based on the nature of support.

Linear sequential model: Drawbacks

- Projects rarely follow complete sequential flow, iterations are involved and changes can cause confusion if not well planned.
- Linear sequential model doesn't work if requirements change often and vital requirements come in later.
- It takes some time for customer/stake holders to look at the working code/product in this model.

Prototyping model



- Useful when customer and/or developer has an idea of the overall objectives but, not clear of specific details.
- Prototype serves as the “first system”.

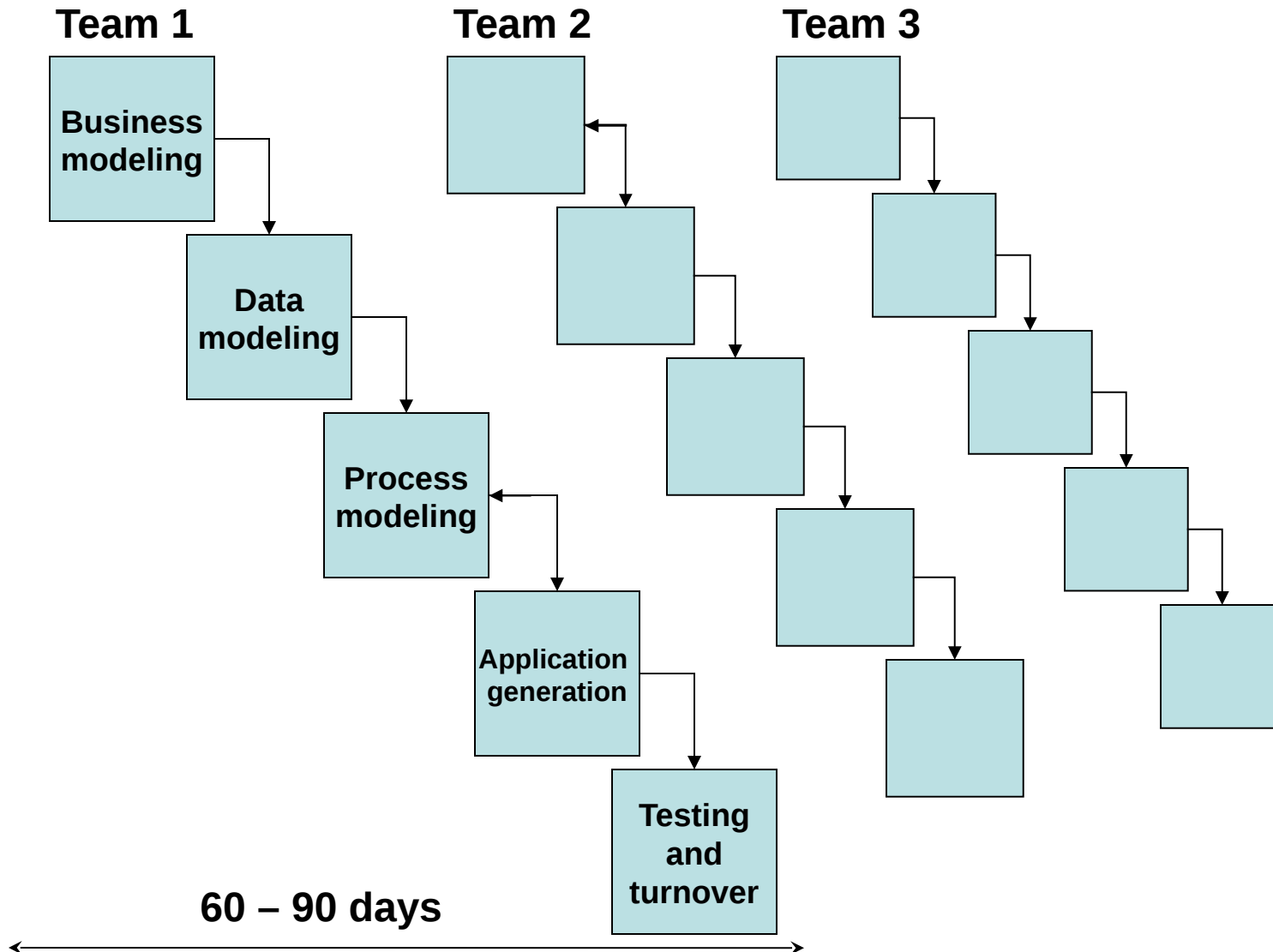
Prototyping model

- Work begins with gathering whatever understanding is available with developers and customers regarding requirements.
- A quick design is done and prototype is done.
- Prototype is evaluated by customer and requirements are refined jointly.
- No focus on efficiency, quality, maintainability etc.

Prototyping model: Drawbacks

- Typically, the prototype needs to be completely discarded and the product needs to be built from scratch.
 - Customer might not be satisfied.
- Compromises are made on design and implementation aspects.
 - Inefficient algorithms, inappropriate OS or choice of language.

Rapid Application Development (RAD)



RAD model

- RAD emphasizes on extremely short development lifecycle.
- Each stage follows the waterfall model.
- Emphasizes on extensive re-use of components, especially in coding and testing phase.

RAD model: Drawbacks

- Not suitable for large projects, needs many members to create many RAD teams.
- Not appropriate for safety-critical systems, systems with a high degree of interoperability, system that are concurrent in nature etc.